Microsoft System Design: Requirements Cheat Sheet

Brief functional and non-functional requirements for eight questions Microsoft candidates have reported. Use them for the first 5 minutes of each practice design; numbers are reasonable assumptions to state out loud, not facts.

1. CDN

Functional

- Serve static content (images, video, JS/CSS) from edge locations near users.
- Fetch from the origin on a cache miss, then cache the response.
- Honor cache rules: TTLs, cache-control headers, query-string handling.
- Purge or invalidate content on demand.
- Route each user to the nearest healthy edge.
- Support custom domains with HTTPS.

Non-functional

- Global scale: hundreds of edge locations, millions of requests/sec.
- Latency: edge hits under ~50 ms for most users.
- Availability: 99.99%+; an edge failure reroutes users automatically.
- Cache hit ratio above 90% to protect origins.
- Purges propagate globally within seconds to a few minutes.
- Security: DDoS absorption, TLS everywhere, optional WAF.

Clarify: static only, or dynamic and video streaming too? Push or pull model?

2. Content sharing via unique URL

Functional

- Users upload content: text, images, video, files.
- Each upload gets a unique, short, shareable URL.
- Anyone with the link (or only permitted users) can view the content.
- Optional expiry and access control: public, private, password-protected.
- Owners can delete or update their content.
- Basic analytics: view counts.

Non-functional

- Read-heavy: ~100:1 views to uploads.
- Scale: ~10M uploads/day, ~1B views/day.
- Low-latency reads worldwide (CDN for media).
- Durability: uploaded content is never lost (replicated blob storage).
- High availability for reads: 99.99%.
- URLs are unique and hard to guess for private content.
- Upload size limit, e.g. 100 MB per file.

Clarify: maximum file size? Do links need to be private by default?

3. VM monitoring system

Functional

- Agents on each VM collect metrics: CPU, memory, disk, network, plus custom metrics.
- Ingest and store metrics as time series.

- Query and graph metrics over time ranges, with aggregation (avg, p95, max).
- Define alert rules (e.g. CPU > 90% for 5 min) and notify via email, pager or webhook.
- Dashboards per VM, per group, per region.
- Optionally feed autoscaling decisions.

Non-functional

- Scale: ~1M VMs × ~100 metrics every 10 s ≈ 10M data points/sec ingested.
- Write-heavy; reads are mostly recent data.
- Alert latency: fire within ~1 minute of the condition.
- Retention: full resolution for ~2 weeks, downsampled for ~1 year.
- Ingestion must not lose data during spikes (buffer with a queue).
- Monitoring must stay up when the monitored systems are failing.
- Multi-tenant isolation: customers see only their own VMs.

Clarify: push (agents send) or pull (collector scrapes)? Logs too, or metrics only?

4. E-commerce system

Functional

- Browse and search the product catalog.
- Add to cart, update cart, check out.
- Place orders and pay through a payment provider.
- Track inventory; prevent selling more than is in stock.
- Order history and status tracking (placed, shipped, delivered).
- User accounts and addresses.

Non-functional

- Scale: ~10M daily users, ~100K orders/day, 10× spikes on sales days.
- Browsing and search: low latency (< 200 ms), eventual consistency is fine.
- Orders, payments, inventory: strongly consistent, no double charges or oversells.
- Idempotent checkout and payment calls.
- Availability: 99.99% for browsing; checkout degrades gracefully, never loses an order.
- Security: PCI compliance for payments, encrypted personal data.

Clarify (this question tests ambiguity): single seller or a marketplace? Which part to go deep on, catalog, cart or checkout?

5. On-prem to cloud migration

Functional

- Move services, databases and storage from on-prem to the cloud.
- Keep every existing feature and API working for clients.
- Migrate data with no loss, including data written during the migration.
- Run on-prem and cloud side by side during the transition (hybrid).
- Shift traffic gradually and roll back at any stage.
- Decommission on-prem once the cloud path is verified.

Non-functional

- Zero or near-zero downtime for customers.
- Data consistency between old and new systems until cutover.
- Same or better latency, availability and security after the move.
- Cost: lower or predictable cloud spend versus on-prem.
- Observability to compare old vs new behavior side by side.

- Compliance: data residency rules per region.

Clarify: lift-and-shift or re-architect? Which parts are stateful? Hard deadline?

6. Message queue

Functional

- Producers publish messages to named topics or queues.
- Consumers receive messages, process them, and acknowledge.
- Unacknowledged messages are redelivered after a timeout.
- Consumer groups: each message goes to one consumer in a group; multiple groups each get a copy.
- Dead-letter queue for messages that keep failing.
- Optional: ordering within a partition, delayed messages, retention and replay.

Non-functional

- Throughput: ~1M messages/sec across the cluster.
- Latency: publish-to-deliver under ~10–50 ms.
- Durability: acknowledged messages survive broker failure (replicated to disk).
- Delivery guarantee: at-least-once by default; state it and make consumers idempotent.
- Horizontal scaling through partitions.
- Availability: 99.99%; broker failover without losing messages.

Clarify: queue (one consumer per message) or pub/sub? Is ordering required? Microsoft has also asked for interfaces and response classes, so be ready to sketch the API in code.

7. Real-time collaborative document editing

Functional

- Create, open and edit documents or spreadsheets.
- Multiple users edit the same document at once and see each other’s changes live.
- Show presence: who is viewing, cursor positions.
- Resolve conflicting edits so everyone converges to the same document.
- Version history and restore.
- Sharing and permissions: owner, editor, viewer.

Non-functional

- Edits appear to other users within ~100–200 ms.
- Up to ~100 concurrent editors per document; millions of documents.
- No lost edits, even with dropped connections (offline edits sync later).
- Strong eventual consistency: all clients converge.
- Durability: every saved change persisted and replicated.
- Availability: 99.99%.

Clarify: text documents or spreadsheets (formulas add dependency recalculation)? Offline editing needed? Key deep dive: operational transformation (OT) vs CRDTs.

8. Cloud file storage (OneDrive-style)

Functional

- Upload, download, delete files and folders.
- Sync files across a user’s devices automatically.
- Share files or folders with other users, with permissions.
- File version history.

- Resumable uploads for large files.
- Notify devices when a file changes.

Non-functional

- Scale: ~100M users, ~10 GB average each ≈ 1 EB stored.
- Durability: 11 nines; files are never lost (replication, erasure coding).
- Large files up to ~50 GB, uploaded in chunks.
- Sync latency: changes appear on other devices within seconds.
- Bandwidth efficiency: sync only changed chunks, deduplicate identical chunks.
- Consistency: last writer wins or conflict copies; never silently lose an edit.
- Availability: 99.9%+; security with encryption at rest and in transit.

Clarify: sync client in scope or web only? Is sharing in scope? Metadata service and chunked block storage are the usual deep dives.